UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

MACROAREA DI SCIENZE MATEMATICHE, FISICHE E NATURALI



CORSO DI STUDIO IN

Informatica

TESI DI LAUREA

A Domain-Specific Language for Security Assurance of 5G SCAS Tests from Network Traffic Captures

Relatrice:

Prof.ssa Paola Vocca

Correlatore:

Dott. Francesco Mancini

Laureando:

Samuele De Santis

Matricola 0348324

Anno Accademico 2025/2026

Indice

1 Introduzione **1**

2 Background **2**

2.1 Architettura 5G e implicazioni per la Security Assurance 2

2.2 Test SCAS ed evidenze di verifica 3

3 Definizione del problema e requisiti **4**

3.1 Formulazione del problema 4

3.2 Dalla specifica SCAS alla rappresentazione dichiarativa 4

3.3 Capacità espressive richieste dalla DSL 5

3.4 Obiettivi e requisiti di sistema 6

3.5 Scope e assunzioni 7

4 Architettura proposta **8**

4.1 Architettura generale 8

4.2 Estrazione strutturata del traffico 9

4.3 Normalizzazione degli eventi 9

4.4 Risoluzione semantica 10

4.5 Selezione dell'evidenza tramite DSL 10

4.6 Verifica 11

4.7 Reporting 11

5 Domain-Specific Language e motore di verifica **13**

5.1 Struttura della grammatica 13

5.2 Rappresentazione interna delle regole 14

5.3 Selezione mediante SEQUENCE 16

5.3.1 Dal pattern all'automa 16

5.3.2 Formalizzazione dell'automa 16

5.3.3 Le ε-transizioni 17

5.3.4 Compilazione del pattern mediante Thompson 17

5.3.5 Funzionamento del matcher 18

5.3.6 Avvio e riavvio dei candidati 19

5.3.7 Produzione e filtraggio dei match 19
5.4 Selezione mediante COLLECT 20
5.5 Verifica statica 20
5.6 Verifica temporale 21
5.7 Determinazione del verdetto 22

6 Testing e Mutation testing **24**
6.1 Obiettivi e metodologia della validazione 24
6.2 Ambiente sperimentale e casi di test 24
6.3 Valutazione delle capacità di verifica 26
6.3.1 AMF Synchronization Failure - Case B 26
6.3.2 UPF TEID Uniqueness 28
6.3.3 AMF RES* Verification Failure - Cases A/B/C/D 28
6.4 Mutation testing 31

7 Discussione, conclusioni e sviluppi futuri **34**

Bibliografia **36**

Glossario e acronimi **38**

Capitolo 1

Introduzione

L'evoluzione delle reti mobili verso la quinta generazione ha introdotto un'architettura più articolata, composta da diverse funzioni di rete che cooperano per erogare i servizi del sistema. In questo contesto, la verifica della sicurezza assume un ruolo importante ed è supportata dalle *Security Assurance Specifications* (SCAS), che definiscono requisiti e procedure di test per valutare specifiche proprietà di sicurezza [1, 2]. Tuttavia, determinare l'esito di una prova a partire dalle catture di traffico non è sempre immediato: i criteri di verifica sono descritti in linguaggio naturale, mentre le evidenze necessarie possono essere distribuite tra messaggi e protocolli differenti.

Il presente lavoro affronta questo problema proponendo un *Domain-Specific Language* (DSL) che permette di esprimere i criteri di verifica in forma dichiarativa, separandoli dalla logica del software che li valuta. Il sistema sviluppato seleziona gli eventi rilevanti dalle catture di traffico, applica le regole specificate e produce un report contenente il verdetto e le evidenze che lo giustificano. La soluzione è stata valutata su diversi casi SCAS e attraverso esperimenti di *mutation testing*, nei quali vengono introdotte alterazioni controllate nelle catture per verificare la capacità del sistema di rilevare le modifiche apportate.

Capitolo 2

Background

2.1 Architettura 5G e implicazioni per la Security Assurance

L'architettura della 5G Core Network adotta un modello fortemente modulare, nel quale le principali funzionalità della rete sono distribuite tra componenti distinti, denominati *Network Functions* (NF) [3]. Ciascuna funzione svolge compiti specifici, come l'autenticazione degli utenti, la gestione della mobilità, la registrazione dei servizi o la gestione delle sessioni.

Le Network Functions del piano di controllo cooperano secondo il paradigma della *Service-Based Architecture* (SBA), attraverso il quale espongono e utilizzano servizi mediante interfacce standardizzate. Tra le principali funzioni coinvolte nei casi di test considerati rientrano l'AMF (*Access and Mobility Management Function*), responsabile della registrazione e della mobilità degli utenti; l'AUSF (*Authentication Server Function*), coinvolta nell'autenticazione; l'UDM (*Unified Data Management*), che gestisce i dati degli abbonati; e l'NRF (*Network Repository Function*), che supporta la registrazione e la scoperta delle Network Functions. Sul piano utente opera invece l'UPF (*User Plane Function*), responsabile dell'instradamento e dell'elaborazione del traffico dati degli utenti [3, 4].

La presenza di funzioni distinte e di numerose interazioni tra esse rende necessario valutare la sicurezza sia dei singoli componenti, sia dei comportamenti osservabili nelle comunicazioni. In questo contesto si inserisce il modello di *Security Assurance* sviluppato da GSMA e 3GPP, che comprende il *Network Equipment Security Assurance Scheme* (NESAS) e le *Security Assurance Specifications* (SCAS).

Il NESAS definisce uno schema di valutazione della sicurezza delle apparecchiature di rete, comprendendo la valutazione dei processi di sviluppo e gestione del prodotto adottati dai fornitori. Le specifiche SCAS, sviluppate dal 3GPP, definiscono invece requisiti di sicurezza e relativi casi di test tecnici per valutare le diverse funzioni di rete, costituendo il riferimento normativo per le verifiche affrontate nel presente lavoro [2, 5].

2.2 Test SCAS ed evidenze di verifica

Le specifiche SCAS descrivono, per ciascuna funzione di rete considerata, una serie di casi di test finalizzati alla verifica di specifiche proprietà di sicurezza. Ogni caso definisce le condizioni e le procedure da seguire, i risultati attesi e le evidenze necessarie a documentare la valutazione [1, 6].

La descrizione di un test case comprende generalmente i seguenti elementi:

- *Test Name:* identificatore e denominazione del caso di test;
- *Purpose:* obiettivo della verifica e proprietà di sicurezza considerata;
- *Pre-Conditions:* condizioni iniziali necessarie all'esecuzione;
- *Execution Steps:* operazioni da eseguire durante il test;
- *Expected Results:* comportamento atteso per il soddisfacimento del requisito;
- *Expected Format of Evidence:* indicazioni sulle evidenze da raccogliere a supporto della valutazione.

A titolo esemplificativo, si consideri il test `TC_DISC_AUTHORIZATION_SLICE_NRF` [7], relativo all'autorizzazione delle richieste di discovery presso l'NRF. Una rappresentazione semplificata della specifica è riportata di seguito.

```
Test Case Name: TC_DISC_AUTHORIZATION_SLICE_NRF

Purpose:
    Verify that the NRF rejects a slice-specific NF discovery request
    when the requested NF instance does not allow discovery
    from the requester's network slice.

Pre-Conditions:
    - NF1 and NF2 are authenticated with the NRF.
    - NF1 belongs to slice B.
    - NF2 allows discovery only from slice A.

Execution Steps
    1. NF2 registers its slice information with the NRF.
    2. NF1 sends an NF discovery request targeting NF2.
    3. The NRF evaluates the slice-specific authorization conditions.

Expected Results:
    The NRF returns a 403 Forbidden response.

Expected Format of Evidence:
    Evidence suitable for the interface
```

Come illustrato nell'esempio, oltre alla procedura e al risultato atteso, la descrizione del test può specificare anche le evidenze da raccogliere, tra cui catture di traffico di rete (PCAP), log applicativi e di sistema e documentazione dei risultati. Le catture PCAP consentono, in particolare, di osservare i messaggi scambiati tra i componenti della rete, i valori dei campi protocollari e le informazioni temporali associate alle comunicazioni.

La disponibilità di tali evidenze permette di documentare il comportamento osservato durante il test, ma non implica che tutte le proprietà di sicurezza siano direttamente osservabili nel traffico acquisito. Le informazioni necessarie alla valutazione dipendono infatti dalla natura del requisito e dalle modalità con cui il test viene eseguito.

Capitolo 3

Definizione del problema e requisiti

3.1 Formulazione del problema

I casi di test SCAS sono espressi attraverso specifiche *human-readable*, destinate quindi alla lettura e all'interpretazione umana. Sebbene questa rappresentazione sia adatta alla documentazione delle procedure e dei comportamenti attesi, i criteri di verifica espressi in linguaggio naturale richiedono una traduzione in una forma strutturata per poter essere elaborati da un motore automatico.

Il problema può quindi essere ricondotto alla seguente domanda: *è possibile esprimere in forma machine-readable la logica necessaria a verificare l'esito di un test SCAS, utilizzando come evidenza il traffico di rete acquisito durante la sua esecuzione?*

3.2 Dalla specifica SCAS alla rappresentazione dichiarativa

Per tradurre i criteri SCAS in regole verificabili, occorre innanzitutto individuare quali aspetti delle procedure di test debbano essere rappresentati e quali informazioni siano necessarie per valutarli.

Quando i criteri di verifica vengono definiti direttamente nel codice applicativo, l'introduzione di un nuovo caso di test può richiedere interventi specifici sulla logica del sistema, anziché limitarsi alla definizione delle condizioni da verificare.

Per separare le caratteristiche proprie del singolo test dalla logica generale del motore di verifica, viene quindi introdotto un *Domain-Specific Language* (DSL), che permette di rappresentare in forma dichiarativa gli eventi da individuare nel traffico osservato e le proprietà che devono essere soddisfatte per determinare l'esito del test.

L'obiettivo non è riprodurre integralmente nella DSL la procedura descritta dalla specifica SCAS, ma esprimere soltanto i criteri necessari alla verifica dell'esito sulla base dell'evidenza disponibile. La variabilità tra test differenti può così essere trasferita, per quanto possibile, dal codice applicativo a regole interpretabili da un motore comune.

3.3 Capacità espressive richieste dalla DSL

Per definire le capacità espressive della DSL sono state analizzate le esigenze di verifica dei casi di test SCAS considerati, individuando gli elementi che devono poter essere rappresentati attraverso il linguaggio. Da questa analisi emergono due esigenze principali:

1. *Selezione dell'evidenza rilevante.* A seconda della natura del test, può essere necessario individuare una sequenza ordinata di eventi oppure raccogliere tutte le occorrenze di una determinata tipologia. Nel primo caso assumono rilevanza l'ordine degli eventi e l'eventuale presenza di sottoflussi opzionali; nel secondo, l'obiettivo è costruire una collezione di evidenze sulla quale valutare proprietà che coinvolgono più occorrenze.
2. *Verifica delle proprietà sull'evidenza selezionata.* Una volta individuata l'evidenza, il linguaggio deve consentire di esprimere i criteri necessari a determinarne l'esito. Tali criteri possono riguardare relazioni tra valori osservati in eventi differenti, confronti con valori attesi e, nei casi in cui il tempo costituisca parte del requisito, vincoli temporali tra eventi osservati.

La Tabella 3.1 mette in relazione ciascuna capacità espressiva con il costrutto introdotto nella DSL e con un caso di test in cui tale capacità viene utilizzata.

Capacità	**Costrutto DSL**	**Test SCAS in cui viene utilizzato**
Riconoscere successioni ordinate di messaggi	`SEQUENCE`	NRF Discovery Authorization; UDM Resynchronization; UDM SUCI De-concealment; AMF Synchronization Failure; AMF RES* Verification Failure
Gestire eventi o sottoflussi opzionali	`OPTIONAL`	AMF RES* Verification Failure
Raccogliere occorrenze dello stesso tipo	`COLLECT`	UPF TEID Uniqueness
Confrontare valori osservati e attesi	`REQUIRE ==, !=`	NRF Discovery Authorization; UDM Resynchronization; AMF Synchronization Failure; AMF RES* Verification Failure; UDM SUCI De-concealment
Verificare l'unicità di valori	`REQUIRE UNIQUE`	UPF TEID Uniqueness
Applicare criteri in base a condizioni	`IF`	AMF RES* Verification Failure
Verificare vincoli temporali	`MFOTL CASE`	AMF Synchronization Failure

Tabella 3.1: Capacità espressive della DSL e relativi casi di test SCAS

La tabella mostra come alcune capacità, quali il riconoscimento di sequenze e il confronto tra valori, siano utilizzate in più casi di test, mentre altre rispondano a esigenze più specifiche.

3.4 Obiettivi e requisiti di sistema

L'obiettivo del sistema è automatizzare la verifica dell'esito dei test SCAS a partire dalle evidenze ricavate dalle catture di traffico, applicando i criteri definiti attraverso il DSL e producendo un verdetto motivato. La separazione tra le regole specifiche dei singoli test e la logica generale di verifica mira inoltre a consentire l'introduzione di nuovi casi attraverso la definizione di nuove specifiche DSL, senza dover sviluppare una procedura di verifica dedicata per ciascuno di essi. L'intento è quindi ridurre per quanto possibile gli interventi sulla logica interna del sistema, con l'obiettivo di rendere sufficiente, per i casi supportati, la sola definizione di una nuova specifica DSL.

I principali requisiti individuati sono riportati nella Tabella 3.2.

ID	Requisito	Descrizione
R1	Genericità	Il motore di verifica non deve dipendere da uno specifico caso di test SCAS.
R2	Riusabilità	I componenti comuni devono poter essere riutilizzati tra test differenti.
R3	Separazione delle responsabilità	La logica relativa all'interpretazione dei protocolli deve essere separata dalla logica specifica del test.
R4	Selezione dell'evidenza	Il sistema deve consentire l'identificazione di eventi, sequenze e collezioni rilevanti ai fini della verifica.
R5	Verifica statica	Il sistema deve poter verificare confronti, condizioni e proprietà definite sui valori osservati.
R6	Verifica temporale	Il sistema deve consentire l'espressione e la valutazione di proprietà temporali.
R7	Gestione dell'evidenza non conclusiva	Il sistema deve distinguere una violazione verificata dall'insufficienza delle evidenze, sia nella selezione sia nella valutazione dei criteri statici.
R8	Tracciabilità	Il verdetto deve essere riconducibile all'evidenza osservata e ai relativi frame della cattura originale.
R9	Riproducibilità	Il sistema deve conservare informazioni sufficienti a ripetere l'analisi nelle medesime condizioni.
R10	Reporting uniforme	Test differenti devono produrre risultati strutturati secondo uno schema comune.
R11	Sensibilità alle variazioni	Il sistema deve essere sensibile ad alterazioni controllate dell'evidenza che modificano il soddisfacimento dei criteri di verifica.

Tabella 3.2: Requisiti del sistema di verifica proposto

Questi requisiti costituiscono il riferimento per le scelte architetturali descritte nel Capitolo 4.

3.5 Scope e assunzioni

Il lavoro considera come sorgente primaria di evidenza le catture di traffico generate durante l'esecuzione dei test; pertanto la possibilità di effettuare una verifica dipende dalla disponibilità e dalla completezza delle informazioni osservabili nel file PCAP.

L'analisi sperimentale è focalizzata sulla *5G Core Network* e sui protocolli coinvolti nei casi di test considerati, e si assume che tali protocolli possano essere correttamente decodificati e trasformati in una rappresentazione strutturata utilizzabile dai componenti successivi della pipeline.

Queste assunzioni definiscono quindi il perimetro entro cui viene valutato il sistema proposto.

Capitolo 4

Architettura proposta

4.1 Architettura generale

L'architettura proposta è organizzata in componenti con responsabilità distinte, progettati per elaborare le evidenze ricavate dalle catture di traffico e valutarle secondo i criteri definiti per ciascun test. Questa suddivisione permette di separare le regole specifiche dei singoli casi dalla logica generale del sistema, favorendo il riutilizzo dei componenti.

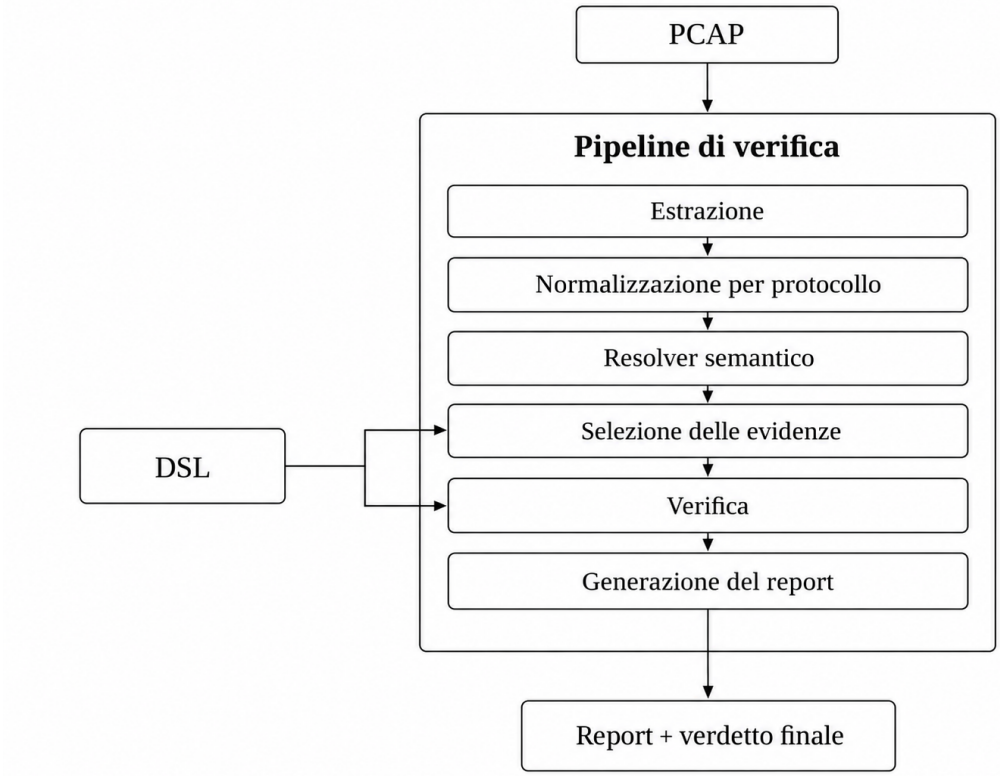


Figura 4.1: Architettura generale della pipeline di verifica degli esiti SCAS.

Il sistema riceve due input principali: una cattura di traffico, contenente le evidenze da analizzare, e una specifica DSL, che definisce quali eventi individuare e quali condizioni verificare. Quest'ultima riveste un ruolo centrale nell'architettura, poiché guida la selezione delle evidenze e la successiva applicazione dei criteri, mentre la pipeline mette a disposizione i meccanismi comuni di elaborazione e verifica.

L'esecuzione è organizzata attraverso il file di configurazione `tests.json`, che associa a ciascun caso di test la cattura, la specifica DSL e i parametri necessari all'analisi. La Figura 4.1 illustra l'organizzazione generale dell'architettura e le relazioni tra i suoi componenti.

4.2 Estrazione strutturata del traffico

La cattura del traffico di rete effettuata durante un test viene conservata in un file PCAP, che costituisce la sorgente delle evidenze osservabili. Poiché le informazioni registrate non sono immediatamente disponibili in una forma adatta all'analisi, è necessaria una fase preliminare di estrazione che le renda accessibili in modo strutturato.

A tale scopo viene utilizzato TShark, un analizzatore di protocolli di rete che applica i *dissector* dei protocolli presenti nella cattura e rende accessibili i relativi campi decodificati. L'estrazione avviene mediante l'opzione `-T ek`, che produce record strutturati in formato JSON, consentendo alla pipeline di operare sui campi decodificati anziché sui byte grezzi dei pacchetti. [8]

I record prodotti contengono informazioni generali, come numero di frame, timestamp e indirizzi IP, insieme ai campi propri dei protocolli analizzati. La diversa organizzazione di questi campi rende necessaria una successiva fase di normalizzazione.

4.3 Normalizzazione degli eventi

La normalizzazione ha lo scopo di ricondurre le informazioni estratte dai diversi protocolli a una rappresentazione comune degli eventi di rete. Per ciascun protocollo supportato, un normalizzatore interpreta i campi disponibili e li organizza in un oggetto denominato `MessageEvent`, che raccoglie sia le informazioni generali dell'evento sia i dati specifici del messaggio.

La struttura del `MessageEvent` comprende i seguenti attributi:

```
class MessageEvent:
    frame_number: int
    timestamp: float
    protocol: str
    kind: str

    src: str | None = None
    dst: str | None = None

    message_type: str | None = None

    related_frames: list[int] = field(default_factory=list)

    correlation: dict[str, Any] = field(default_factory=dict)

    fields: dict[str, Any] = field(default_factory=dict)
```

Nell'implementazione attuale sono previsti normalizzatori per NAS, HTTP/2 e PFCP.

4.4 Risoluzione semantica

La normalizzazione produce eventi rappresentati secondo una struttura comune, ma in alcuni casi il loro significato applicativo non è ancora completamente esplicito. Ciò avviene soprattutto per messaggi relativi alle interazioni della Service-Based Architecture, dove i messaggi HTTP/2 possono rappresentare operazioni differenti in funzione del servizio richiesto e del contenuto applicativo.

Per gestire questa esigenza, viene introdotto un *resolver semantico*, incaricato di interpretare gli eventi normalizzati e associarli a tipologie coerenti con il dominio 5G. Ad esempio, analizzando il percorso della richiesta HTTP/2, il resolver può riconoscere un'operazione di *NF Discovery* presso l'NRF e classificare gli eventi correlati come `nrf.discovery.request` e `nrf.discovery.response`.

Il resolver opera quindi a un livello di astrazione superiore rispetto ai normalizzatori, in quanto non ricostruisce nuovamente i messaggi, ma utilizza le informazioni già disponibili per completarne l'interpretazione e rendere esplicito il tipo semantico degli eventi.

4.5 Selezione dell'evidenza tramite DSL

La normalizzazione e l'eventuale risoluzione semantica trasformano il traffico acquisito in una sequenza di eventi strutturati, tra i quali la fase di selezione individua quelli richiesti dal caso di test considerato.

Per descrivere le evidenze da individuare, il DSL mette a disposizione due costrutti principali: `SEQUENCE`, che permette di specificare una successione ordinata di eventi, e `COLLECT`, utilizzato per raccogliere più occorrenze di un determinato tipo di evento. Attraverso questi costrutti è possibile indicare quali eventi ricercare nella cattura e, quando necessario, le relazioni di ordine che devono sussistere tra essi.

Gli eventi individuati vengono associati a identificatori simbolici, denominati *alias*, definiti nella specifica DSL. Un alias può rappresentare, ad esempio, un particolare messaggio riconosciuto tramite `SEQUENCE` oppure una collezione di messaggi ottenuta mediante `COLLECT`. L'associazione tra l'alias e l'evidenza corrispondente prende il nome di *binding*.

I binding prodotti dalla selezione vengono raccolti nel `BindingContext`, una struttura che rende disponibili le evidenze individuate attraverso i rispettivi alias. I criteri di verifica possono così riferirsi direttamente agli eventi selezionati, indipendentemente dalla loro posizione nella cattura.

4.6 Verifica

Una volta costruito il `BindingContext`, la fase di verifica valuta l'evidenza selezionata rispetto ai criteri definiti nella DSL, al fine di stabilire se il comportamento osservato soddisfi le condizioni previste per il caso di test.

L'architettura prevede due modalità complementari: la verifica statica e la verifica temporale.

La prima opera sui valori associati ai binding, valutando proprietà e relazioni tra i dati estratti dagli eventi selezionati. È utilizzata, ad esempio, per controllare la corrispondenza tra valori o l'assenza di duplicati.

La seconda riguarda invece le relazioni temporali tra gli eventi, come il rispetto di determinati intervalli di tempo. Tali proprietà sono espresse mediante la *Metric First-Order Temporal Logic* (MFOTL) e valutate attraverso il motore di verifica esterno MonPoly.

I risultati ottenuti dalle due modalità concorrono alla determinazione del verdetto complessivo del caso di test.

4.7 Reporting

L'ultima fase dell'architettura riguarda la produzione del report, che affianca al verdetto le informazioni su cui esso si basa. Il reporter, comune ai diversi casi di test, organizza tali informazioni in un documento strutturato in formato `JSON`, distinguendo i dati relativi al test e all'esecuzione dai criteri valutati, dalle evidenze osservate e dal risultato complessivo.

Il seguente schema illustrativo semplificato presenta i principali campi del report; i segnaposto indicano parti omesse e non costituiscono un documento JSON completo:

```
{
    "metadata": {
        "generator": "generic_verification_pipeline",
        "tshark_version": "..."
    },
    "test": { "id": "..." },
    "source": { "pcap": "...", "sha256": "..." },
    "configuration": {
        "pipeline": "...", "rules": "...", "traffic": { }
    },
    "criteria": [ ... ],
    "verification": {
        "results": [ ... ],
        "overall_verdict": "PASS"
    },
    "evidence": { "bindings": { ... } },
    "explanation": { "summary": "..." },
    "verdict": "PASS"
}
```

Il report favorisce sia la verificabilità sia la riproducibilità dell'analisi. I criteri valutati e i riferimenti alle evidenze consentono di controllare le motivazioni del verdetto, mentre i metadati relativi alla cattura, alla configurazione e alla versione di TShark documentano alcune condizioni utili per ripetere la verifica. In particolare, l'hash SHA-256 del file PCAP permette di controllare che una successiva analisi utilizzi la stessa cattura.

Capitolo 5

Domain-Specific Language e motore di verifica

5.1 Struttura della grammatica

La sintassi del DSL è definita mediante una grammatica realizzata con Lark, una libreria Python per la costruzione di parser a partire da grammatiche formali. Le regole del linguaggio utilizzano i tipi semantici degli eventi e i relativi attributi, resi disponibili dalle fasi di normalizzazione e risoluzione semantica.

Una specifica è composta da istruzioni dedicate alla selezione delle evidenze e alla definizione dei criteri di verifica. La selezione può essere espressa attraverso `SEQUENCE`, che descrive una successione ordinata di eventi, oppure `COLLECT`, che raccoglie le occorrenze di un determinato tipo di evento. In entrambi i casi, la parola chiave `AS` permette di associare un identificatore simbolico, denominato alias, all'evidenza individuata. All'interno di `SEQUENCE` è inoltre possibile utilizzare `OPTIONAL` per indicare porzioni della sequenza la cui presenza non è obbligatoria.

I criteri di verifica sono definiti mediante `REQUIRE`, che consente di esprimere confronti tra valori letterali e campi degli eventi selezionati. Il linguaggio prevede inoltre `REQUIRE UNIQUE` per verificare proprietà di unicità e `IF` per applicare determinate regole soltanto quando una condizione è soddisfatta. Le proprietà temporali sono invece espresse mediante MFOTL.

Le principali produzioni della grammatica sono riportate di seguito.

```
start: statement+

?statement: sequence_rule
          | collect_rule
          | rule

?rule: static_rule
     | temporal_rule

?static_rule: comparison_rule
            | unique_rule
            | if_rule

sequence_rule: "SEQUENCE" "{" sequence_element+ "}"
?sequence_element: sequence_item | optional_block
sequence_item: FIELD "AS" NAME
optional_block: "OPTIONAL" "{" sequence_element+ "}"

collect_rule: "COLLECT" FIELD "AS" NAME

comparison_rule: "REQUIRE" FIELD OP operand
unique_rule: "REQUIRE" "UNIQUE" FIELD
if_rule: "IF" FIELD OP operand "{" rule+ "}"

temporal_rule: "MFOTL" "CASE" VALUE MFOTL_BLOCK

?operand: FIELD -> field_operand
        | VALUE -> literal_operand
```

I riferimenti agli eventi e ai relativi attributi sono rappresentati attraverso `FIELD`, che utilizza una notazione gerarchica separata da punti. Nella parte dedicata alla selezione, ad esempio, è possibile dichiarare:

```
SEQUENCE {
    ausf.authentication.response AS initial_vector
    nas.authentication_request AS auth_request
}
```

In questo esempio, i due eventi semantici vengono associati rispettivamente agli alias `initial_vector` e `auth_request`, attraverso i quali è possibile accedere ai relativi campi nelle successive regole di verifica:

```
REQUIRE auth_request.rand == initial_vector.rand
```

`NAME` identifica gli alias introdotti dalla specifica, mentre `VALUE` rappresenta valori letterali utilizzabili nelle espressioni.

5.2 Rappresentazione interna delle regole

Le istruzioni scritte nel DSL devono essere interpretate dal sistema per poter essere utilizzate durante la selezione e la verifica delle evidenze. A questo scopo, il parser costruito con Lark analizza la specifica e ne riconosce le strutture sintattiche, che vengono successivamente trasformate in rappresentazioni interne attraverso un `Transformer`.

Il matcher e i verificatori non operano quindi direttamente sul testo della DSL, ma su rappresentazioni strutturate che conservano esclusivamente le informazioni necessarie all'elaborazione.

A seconda dello statement considerato, vengono mantenute informazioni differenti, quali il tipo di operatore, i riferimenti ai campi, gli alias, gli operandi di confronto, eventuali regole annidate e, nel caso di proprietà temporali, la relativa formula.

Ad esempio, una regola come

```
REQUIRE auth_request.rand == initial_vector.rand
```

viene trasformata concettualmente in una struttura del tipo:

```
field: auth_request.rand
operator: ==
expected:
    type: field
    value: initial_vector.rand
```

In questo modo viene mantenuta anche la distinzione tra un operando costituito da un valore letterale e un operando che rappresenta invece il riferimento a un altro campo della specifica.

Per il costrutto `SEQUENCE`, la rappresentazione ottenuta dal parsing viene ulteriormente trasformata in un *Abstract Syntax Tree* (AST) dedicato alla descrizione del pattern sequenziale. I tipi di nodo attualmente necessari sono:

- `Sequence`, che rappresenta l'insieme ordinato degli elementi del pattern;
- `Atom`, associato a un singolo tipo di evento atteso e al relativo alias;
- `Optional`, utilizzato per racchiudere elementi la cui presenza è opzionale.

Ad esempio, la specifica

```
SEQUENCE {
    event.a AS a
    OPTIONAL {
        event.b AS b
    }
    event.c AS c
}
```

può essere rappresentata concettualmente attraverso il seguente albero:

```
Sequence
├── Atom(event.a, a)
├── Optional
│   └── Atom(event.b, b)
└── Atom(event.c, c)
```

È importante precisare che non tutti gli elementi della DSL vengono trasformati nello stesso AST. La struttura composta da `Sequence`, `Atom` e `Optional` è specificamente utilizzata per rappresentare i pattern sequenziali, mentre le regole statiche e temporali vengono mantenute in strutture interne adatte ai rispettivi meccanismi di verifica.

Le rappresentazioni ottenute costituiscono quindi il livello di collegamento tra il parsing della specifica e i componenti incaricati della sua elaborazione.

Prima di procedere alla selezione dell'evidenza, viene inoltre effettuata una validazione strutturale degli alias utilizzati nei criteri statici. Questo controllo è necessario per verificare la coerenza dei riferimenti presenti nella specifica DSL, evitando che un alias utilizzato nelle regole di verifica, ma mai dichiarato, venga erroneamente interpretato come un'evidenza mancante durante l'analisi della cattura.

La validazione viene effettuata raccogliendo tutti gli alias dichiarati nella fase di selezione, a cui vengono aggiunti gli eventuali alias esterni, di configurazione o derivati. Questi vengono confrontati con gli alias utilizzati nelle regole di verifica, comprese le condizioni e le regole annidate.

La validazione permette quindi di individuare gli alias utilizzati senza essere stati dichiarati, distinguendo gli errori presenti nella specifica dai casi in cui un alias correttamente dichiarato non risulti associato ad alcuna evidenza durante la selezione.

5.3 Selezione mediante SEQUENCE

Questa sezione descrive il meccanismo utilizzato per individuare, all'interno del traffico osservato, le sequenze di eventi.

5.3.1 Dal pattern all'automa

Per il riconoscimento dei pattern sequenziali, il sistema utilizza un automa a stati finiti non deterministico (*Nondeterministic Finite Automaton*, NFA). Il pattern, precedentemente rappresentato mediante un AST, viene trasformato in un NFA che descrive i possibili percorsi di riconoscimento degli eventi.

Questa scelta permette di utilizzare un unico meccanismo di ricerca per sequenze con strutture differenti, comprese quelle contenenti blocchi `OPTIONAL`, senza dover sviluppare una logica di riconoscimento specifica per ciascun pattern.

5.3.2 Formalizzazione dell'automa

Formalmente, l'NFA adottato è definito dalla seguente quintupla:

$$A = \langle \Sigma,\ Q,\ q_0,\ Q_F,\ \delta \rangle$$

dove

- Σ è l'alfabeto, costituito dall'insieme dei tipi semantici di evento riconosciuti dall'automa;
- Q è l'insieme finito degli stati;
- $q_0 \in Q$ è lo stato iniziale;
- $Q_F \subseteq Q$ è l'insieme degli stati finali. Nell'implementazione proposta viene utilizzato un unico stato di accettazione, pertanto $Q_F = \{q_f\}$;
- δ è la funzione di transizione non deterministica, definita come

$$\delta : Q \times (\Sigma \ \cup \ \{\varepsilon\}) \to \mathcal{P}(Q)$$

dove $\mathcal{P}(Q) = 2^Q$ indica l'insieme delle parti di Q, ossia l'insieme di tutti i suoi sottoinsiemi.

5.3.3 Le ε-transizioni

Una transizione ordinaria, etichettata con un simbolo $a \in \Sigma$, può essere attraversata soltanto quando l'evento corrente della traccia presenta il tipo semantico richiesto. Il suo attraversamento comporta il consumo dell'evento e l'avanzamento nella lettura della traccia. Tale transizione è rappresentata come:

$$q_i \xrightarrow{a} q_j$$

Una transizione ε, invece, ha la forma:

$$q_i \xrightarrow{\varepsilon} q_j$$

e permette di passare da uno stato all'altro senza consumare alcun evento, ossia senza avanzare nella lettura della traccia.

Le ε-transizioni permettono quindi di collegare stati e percorsi dell'automa senza richiedere il riconoscimento di nuovi eventi. Questa caratteristica viene utilizzata nella costruzione di Thompson per combinare i diversi frammenti che compongono il pattern.

5.3.4 Compilazione del pattern mediante Thompson

Una volta ottenuto l'AST, il pattern sequenziale viene compilato in un NFA mediante la costruzione di Thompson [9]. Tale procedimento si basa su un approccio composizionale: a ciascun nodo dell'albero viene associato un frammento dell'automa, dotato di uno stato iniziale e uno finale; i frammenti vengono successivamente collegati per ottenere l'automa corrispondente alla sequenza completa.

La costruzione dipende dal tipo di nodo considerato:

- `Atom`: genera un frammento costituito da due stati collegati da una transizione etichettata con il tipo semantico dell'evento atteso.
- `Sequence`: combina i frammenti dei propri elementi nell'ordine specificato, collegando lo stato finale di ciascun frammento allo stato iniziale del successivo mediante una transizione ε.
- `Optional`: il blocco viene rappresentato mediante due percorsi alternativi collegati da transizioni ε: uno attraversa il frammento corrispondente agli eventi opzionali, mentre l'altro consente di saltarlo senza consumare tali eventi.

Considerando il pattern sequenziale già introdotto nella Sezione 5.2, la Figura 5.1 ne mostra una rappresentazione schematica mediante NFA, ottenuta applicando la costruzione di Thompson ai nodi dell'AST.

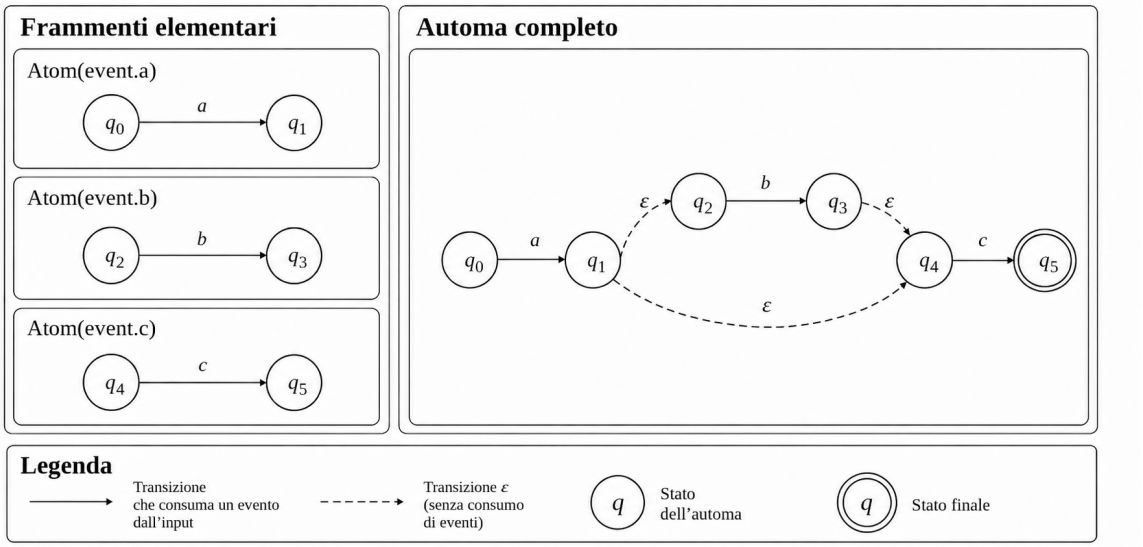


Figura 5.1: Rappresentazione schematica semplificata di un NFA per una sequenza contenente un blocco opzionale.

5.3.5 Funzionamento del matcher

Il riconoscimento delle sequenze di eventi all'interno della traccia è affidato al matcher, il componente che utilizza l'NFA per individuare le corrispondenze con il pattern definito nella DSL. Il matcher riceve in ingresso l'automa e la sequenza ordinata di `MessageEvent`, esaminando gli eventi per individuare i percorsi che consentono di raggiungere uno stato accettante.

Durante questa operazione non è sufficiente conoscere lo stato raggiunto nell'automa, poiché è necessario conservare anche i riferimenti agli eventi che hanno contribuito al riconoscimento. A questo scopo viene introdotta una struttura denominata `ActiveConfiguration`, formalizzabile come:

$$C = \langle q, B \rangle$$

dove

- $q \in Q$ è lo stato corrente dell'automa;
- B è una funzione parziale che associa gli alias definiti nella DSL ai `MessageEvent` già riconosciuti.

Il matcher può mantenere contemporaneamente più configurazioni attive, ciascuna caratterizzata dal proprio stato e dai binding progressivamente costruiti.

Poiché l'automa può contenere transizioni ε, per ogni stato corrente devono essere considerati anche quelli raggiungibili senza consumare ulteriori eventi. A tale scopo viene utilizzata una ε-closure, definita formalmente come:

$$\varepsilon\text{-closure}(q) = \{p \in Q \mid q \xrightarrow{\varepsilon^*} p\}$$

La chiusura comprende tutti gli stati raggiungibili da q attraverso zero o più transizioni ε, incluso lo stesso q. L'espansione di una configurazione attraverso tali transizioni mantiene invariati i binding già acquisiti, poiché non viene riconosciuto alcun nuovo evento.

Durante la scansione della traccia, il matcher confronta il `message_type` di ciascun evento con quello richiesto dalle transizioni uscenti dagli stati delle configurazioni attive. Quando viene individuata una transizione compatibile, la configurazione avanza verso lo stato di destinazione e il `MessageEvent` osservato viene associato all'alias definito nel corrispondente `Atom`. I binding vengono quindi aggiornati e vengono considerate le ulteriori transizioni ε raggiungibili dal nuovo stato.

Una caratteristica importante è che la selezione non richiede che gli eventi previsti dal pattern siano consecutivi nella traccia. Quando un evento non può essere consumato da una determinata configurazione, quest'ultima può rimanere attiva, consentendo di proseguire la ricerca con gli eventi successivi.

La possibilità di mantenere più configurazioni attive consente inoltre al matcher di seguire simultaneamente le alternative previste dall'NFA in presenza di blocchi `OPTIONAL`, comprese le configurazioni raggiungibili mediante ε-transizioni che consentono di omettere gli eventi opzionali.

5.3.6 Avvio e riavvio dei candidati

All'interno di una cattura possono essere presenti più occorrenze degli eventi descritti dalla stessa `SEQUENCE`, anche in posizioni differenti della traccia. Il matcher esamina quindi gli eventi in ordine e, per ogni posizione compatibile con l'inizio del pattern, avvia un nuovo candidato di riconoscimento. Ciascun candidato rappresenta un tentativo indipendente di completare la sequenza a partire dalla posizione individuata e può mantenere più configurazioni attive, corrispondenti ai diversi percorsi dell'automa.

Durante l'elaborazione viene applicata una politica di riavvio (*restart policy*), finalizzata a evitare che eventi appartenenti a tentativi distinti vengano associati alla stessa sequenza. Se viene incontrato un evento compatibile con l'inizio di un nuovo candidato, ma che non può essere consumato da nessuna delle configurazioni attive del candidato corrente, quest'ultimo viene interrotto. L'evento potrà comunque essere considerato come punto di partenza di un candidato distinto.

5.3.7 Produzione e filtraggio dei match

Un candidato completa il riconoscimento quando almeno una delle sue configurazioni raggiunge lo stato accettante dell'NFA. Per ciascun candidato viene considerato il primo punto di completamento, nel quale possono tuttavia essere presenti più configurazioni accettanti, associate a percorsi differenti dell'automa.

A ogni configurazione accettante corrisponde un oggetto `Match`, che conserva la posizione iniziale del candidato (`start_index`), quella in cui è stato completato il riconoscimento (`end_index`) e i binding degli eventi individuati. Il matcher può quindi produrre più match, anche a partire da uno stesso candidato.

La presenza di percorsi alternativi, in particolare quelli introdotti da `OPTIONAL`, può produrre match riferiti allo stesso intervallo della traccia ma contenenti binding differenti. Per eliminare i risultati meno completi senza perdere evidenze distinte, viene applicato un criterio di dominanza: un match domina un altro quando sono soddisfatte tutte le seguenti condizioni.

- i due match condividono `start_index` e `end_index`;
- gli alias del secondo costituiscono un sottoinsieme proprio di quelli del primo;
- per tutti gli alias comuni, le associazioni fanno riferimento agli stessi eventi osservati.

Il filtraggio elimina quindi soltanto i match dominati, conservando quelli che presentano associazioni differenti o che non sono confrontabili secondo tali condizioni.

Al termine del filtraggio, il matcher restituisce i match completi non dominati, che costituiscono il risultato della selezione sequenziale e contengono le associazioni tra gli alias e gli eventi riconosciuti.

5.4 Selezione mediante COLLECT

La selezione mediante `COLLECT` viene effettuata confrontando il tipo semantico dichiarato nella regola con il campo `message_type` degli eventi disponibili. Tutte le occorrenze corrispondenti vengono raccolte e associate all'alias specificato, senza richiedere la costruzione di un automa.

Una regola di raccolta può essere espressa come segue:

```
COLLECT pfcp.session_establishment_response AS responses
```

In questo esempio, `pfcp.session_establishment_response` identifica il tipo di evento da ricercare, mentre `responses` è l'alias assegnato alla collezione risultante.

Gli eventi individuati vengono associati all'alias all'interno del `BindingContext`. A differenza di `SEQUENCE`, dove ciascun alias identifica un singolo evento riconosciuto, nel caso di `COLLECT` il valore associato è una lista di `MessageEvent`, rappresentabile come:

$$B(\texttt{responses}) = [e_1, e_2, e_3, ..., e_n]$$

La struttura del `BindingContext` rimane quindi invariata, mentre cambia la natura dell'evidenza associata all'alias. Le due modalità di selezione, pur adottando meccanismi differenti, producono così una rappresentazione comune dei risultati.

5.5 Verifica statica

Il verificatore statico utilizza i binding presenti nel `BindingContext` per valutare i criteri definiti nelle regole DSL. Per effettuare le verifiche, riceve il contesto prodotto dalla selezione e le regole ottenute durante il parsing della specifica.

Le regole di confronto sono introdotte dalla parola chiave `REQUIRE` e possono essere espresse nella forma generale:

```
REQUIRE alias.campo OP valore
```

dove OP identifica l'operatore di confronto, mentre l'operando a destra può essere un valore letterale oppure un riferimento a un altro campo.

Per effettuare la valutazione, il verificatore deve innanzitutto risolvere i riferimenti presenti nella regola. Nel caso di un binding associato a un singolo evento, viene individuato l'alias interno del `BindingContext` e recuperato il relativo `MessageEvent`. Il percorso gerarchico specificato nella DSL viene quindi utilizzato per accedere ai campi, anche annidati, contenuti nell'attributo `fields` dell'evento. Quando invece il binding contiene una collezione, il campo richiesto viene recuperato da ciascuna evidenza, conservando anche il riferimento ai frame da cui proviene. Il medesimo meccanismo consente inoltre di accedere a informazioni esterne o derivate, eventualmente aggiunte al `BindingContext`, attraverso gli stessi riferimenti gerarchici utilizzati per gli eventi selezionati.

Una volta risolti i riferimenti presenti nella regola, il verificatore dispone dei valori necessari per applicare il criterio richiesto. La valutazione può consistere nel confronto tra due valori oppure nel controllo dell'unicità degli elementi di una collezione. In quest'ultimo caso, vengono considerate anche informazioni strutturate o contenenti più valori, conservando i riferimenti ai frame degli eventuali duplicati individuati.

La valutazione può inoltre dipendere da una condizione definita mediante `IF`. Quando la condizione è soddisfatta, il verificatore esegue le regole contenute nel blocco; in caso contrario, queste non vengono valutate e il blocco assume lo stato `SKIPPED`. Tale stato indica che le verifiche condizionali non sono applicabili e non rappresenta un ulteriore verdetto del test.

Ogni criterio valutato produce un risultato strutturato che associa l'esito alle informazioni utilizzate nella verifica, come i valori osservati e attesi e, quando pertinenti, i riferimenti ai frame. Questi risultati costituiscono la base per la successiva determinazione del verdetto complessivo.

5.6 Verifica temporale

La verifica temporale permette di valutare proprietà che dipendono non soltanto dai valori contenuti nei messaggi, ma anche dalle relazioni temporali tra gli eventi osservati. A questo scopo viene utilizzata la *Metric First-Order Temporal Logic* (MFOTL) [10], un formalismo che consente di esprimere proprietà mediante predicati e operatori temporali, associando a questi ultimi intervalli entro cui devono essere soddisfatte determinate condizioni.

Le formule vengono valutate attraverso MonPoly [11], un motore di monitoraggio esterno che verifica proprietà MFOTL su tracce temporizzate. La formula descrive quindi la proprietà richiesta, mentre MonPoly ne valuta il soddisfacimento rispetto agli eventi osservati.

All'interno del DSL, una proprietà temporale può essere dichiarata mediante il costrutto `MFOTL CASE`. Il seguente esempio, tratto dal test AMF Synchronization Failure – Case A, richiede che a una richiesta di risincronizzazione segua una risposta AUSF entro un intervallo temporale configurato.

```
MFOTL CASE A """
ResynchronizationRequest()
IMPLIES
EVENTUALLY[0,{TIMER_MS}] AusfAuthenticationResponse()
"""
```

Per valutare la formula, MonPoly richiede una traccia temporizzata, nella quale gli eventi sono rappresentati mediante predicati associati ai rispettivi timestamp. Il sistema costruisce questa traccia a partire dagli eventi riconosciuti durante la selezione, utilizzando la configurazione del test per stabilire la corrispondenza tra gli alias della DSL e i predicati della formula.

La stessa configurazione indica il file di *signature*, nel quale sono dichiarati i nomi dei predicati e gli eventuali argomenti che MonPoly può riconoscere. Nell'esempio precedente, tra i predicati utilizzati figurano `ResynchronizationRequest()` e `AusfAuthenticationResponse()`. Prima dell'esecuzione, il parametro `{TIMER_MS}` viene sostituito con il valore configurato per il test, espresso in millisecondi.

Una volta preparate la formula, la signature e la traccia temporizzata, il sistema avvia MonPoly per effettuare la verifica. Nell'implementazione sviluppata viene utilizzata l'opzione `-negate`, che consente di monitorare la negazione della proprietà specificata e quindi di individuare le eventuali violazioni della formula originale. L'output prodotto da MonPoly viene infine interpretato dal verificatore per determinare l'esito del controllo temporale.

5.7 Determinazione del verdetto

Il risultato della verifica di un caso di test è espresso attraverso uno dei tre verdetti previsti dal sistema: `PASS`, quando i criteri richiesti risultano soddisfatti; `FAIL`, quando viene rilevata una violazione; e `INCONCLUSIVE`, quando le evidenze disponibili non consentono di determinare un esito conclusivo.

Il verdetto `INCONCLUSIVE` viene prodotto quando le evidenze disponibili non consentono di valutare in modo conclusivo i criteri del caso di test. Questa situazione può verificarsi già durante la selezione: nel caso di `SEQUENCE`, quando non viene individuato alcun match completo oppure quando, anche dopo il filtraggio, rimangono più corrispondenze complete; nel caso di `COLLECT`, quando non sono presenti eventi del tipo semantico richiesto. L'impossibilità di concludere la verifica può emergere anche successivamente, qualora un criterio statico applicabile richieda un alias al quale non è stato associato alcun evento, come può accadere per un blocco `OPTIONAL` non attraversato durante il riconoscimento. In quest'ultimo caso, il verificatore non dispone dei valori necessari per valutare la regola: l'assenza dell'evidenza richiesta non permette di stabilire né il soddisfacimento né la violazione del criterio.

Quando il caso di test prevede più criteri di verifica, i rispettivi risultati vengono combinati secondo una logica a tre valori, riportata nella Tabella 5.1.

Criterio A	**Criterio B**	**Risultato**
PASS	PASS	PASS
PASS	FAIL	FAIL
PASS	INCONCLUSIVE	INCONCLUSIVE
FAIL	PASS	FAIL
FAIL	FAIL	FAIL
FAIL	INCONCLUSIVE	FAIL
INCONCLUSIVE	PASS	INCONCLUSIVE
INCONCLUSIVE	FAIL	FAIL
INCONCLUSIVE	INCONCLUSIVE	INCONCLUSIVE

Tabella 5.1: Tabella di aggregazione dei risultati di verifica secondo una logica a tre valori.

Come mostrato nella Tabella 5.1, il verdetto complessivo è `PASS` quando tutti i criteri applicabili risultano soddisfatti e `FAIL` quando almeno uno di essi rileva una violazione. `INCONCLUSIVE` viene invece prodotto quando non sono state rilevate violazioni, ma almeno un criterio non può essere valutato. Le regole contrassegnate come `SKIPPED` sono escluse dall'aggregazione, poiché non applicabili.

La stessa logica viene utilizzata quando il caso di test prevede sia verifiche statiche sia temporali, combinando i rispettivi risultati per determinare un unico verdetto complessivo.

Capitolo 6

Testing e Mutation testing

6.1 Obiettivi e metodologia della validazione

La validazione sperimentale comprende sia i casi di test SCAS che hanno guidato lo sviluppo iniziale del sistema, sia ulteriori casi introdotti successivamente per ampliare e valutare le funzionalità supportate. Gli esperimenti considerano procedure relative a differenti funzioni della 5G Core Network, utilizzando come evidenza le informazioni ricavate dalle catture di traffico.

Sono inoltre previsti esperimenti di *mutation testing* [12], finalizzati a valutare la sensibilità del sistema ad alterazioni controllate dell'evidenza. La metodologia adottata e i relativi risultati sono approfonditi nella Sezione 6.4.

6.2 Ambiente sperimentale e casi di test

Per la validazione sono stati utilizzati i seguenti ambienti e strumenti:

- Open5GS [13], implementazione open-source della 5G Core Network che fornisce le principali Network Functions previste dall'architettura 5G;
- free5GC [14], implementazione open-source della 5G Core Network, utilizzabile per lo studio e la sperimentazione delle procedure e delle interazioni tra Network Functions;
- ScasDK [15], piattaforma per l'esecuzione automatizzata di test di sicurezza SCAS, impiegata come riferimento sperimentale; alcuni suoi componenti sono stati adattati per le prove relative a NRF e UDM.

Il sistema proposto opera su catture di traffico già disponibili. L'esecuzione delle procedure SCAS e la generazione del traffico rimangono quindi distinte dalla verifica dell'esito e non rientrano tra le funzionalità della pipeline sviluppata, in accordo con il perimetro definito nella Sezione 3.5.

I casi di test relativi alle funzioni NRF, UDM, AMF e UPF, definiti nelle rispettive specifiche SCAS [7, 16, 17, 18], sono riepilogati nelle Tabelle 6.1 e 6.2, organizzate in base all'implementazione della 5G Core Network da cui provengono le catture analizzate. Per ciascun caso vengono riportati il criterio di verifica applicato e il verdetto ottenuto.

Caso di test	**Criterio di verifica**	**Verdetto**
NRF Discovery Authorization	Risposta HTTP 403	FAIL
UDM Resynchronization	Coerenza dei parametri di autenticazione	PASS
UDM SUCI De-concealment	SUPI restituito uguale al valore atteso	PASS
AMF Sync Failure – Case A	Coerenza dei messaggi e vincolo T3520	PASS
AMF Sync Failure – Case B	Coerenza dei messaggi e vincolo T3520	PASS
UPF TEID Uniqueness	Assenza di F-TEID duplicati	PASS
AMF RES* – Case A	Gestione del fallimento di autenticazione	PASS
AMF RES* – Case B	Gestione del fallimento di autenticazione	INCONCLUSIVE
AMF RES* – Case C	Gestione del fallimento di autenticazione	INCONCLUSIVE
AMF RES* – Case D	Gestione del fallimento di autenticazione	INCONCLUSIVE

Tabella 6.1: Casi di test SCAS e verdetti ottenuti su Open5GS.

Caso di test	**Criterio di verifica**	**Verdetto**
AMF Sync Failure – Case A	Coerenza dei messaggi e vincolo T3520	PASS
AMF Sync Failure – Case B	Coerenza dei messaggi e vincolo T3520	PASS
AMF RES* – Case A	Gestione del fallimento di autenticazione	PASS
AMF RES* – Case B	Gestione del fallimento di autenticazione	PASS
AMF RES* – Case C	Gestione del fallimento di autenticazione	PASS
AMF RES* – Case D	Gestione del fallimento di autenticazione	PASS

Tabella 6.2: Casi di test SCAS e verdetti ottenuti su free5GC.

I verdetti riportati rappresentano l'esito della verifica dei criteri dichiarati rispetto all'evidenza osservata. Un risultato `FAIL` non indica quindi necessariamente un malfunzionamento della pipeline, ma può corrispondere alla corretta rilevazione di un criterio non soddisfatto. È il caso del test NRF, nel quale la risposta osservata presenta uno status HTTP `200`, mentre la regola DSL richiede `403`.

6.3 Valutazione delle capacità di verifica

In questa sezione vengono approfonditi tre gruppi di casi di test, selezionati per esaminare il comportamento delle principali capacità di verifica implementate.

6.3.1 AMF Synchronization Failure - Case B

L'analisi di `AMF Synchronization Failure - Case B` coinvolge tre componenti del sistema: la selezione mediante `SEQUENCE`, la verifica statica sui campi degli eventi e la verifica temporale attraverso MonPoly.

Il caso riguarda la gestione di un fallimento di sincronizzazione durante la procedura di autenticazione 5G AKA. A seguito della ricezione di un'`Authentication Request`, l'UE segnala un `Authentication Failure` con causa di sincronizzazione e fornisce il parametro `AUTS` necessario alla risincronizzazione. L'AMF inoltra quindi le informazioni necessarie all'AUSF. Il timer NAS T3520, avviato dall'UE a seguito dell'invio dell'`Authentication Failure`, ha un valore di 15 secondi nelle condizioni ordinarie di accesso considerate [19]; tale intervallo è utilizzato come parametro di riferimento per il criterio temporale configurato nel test. Nel Case B, il criterio temporale verifica che, nell'intervallo configurato a partire dal fallimento di sincronizzazione, non venga osservata una nuova `Authentication Request` prima della risposta AUSF successiva alla richiesta di risincronizzazione.

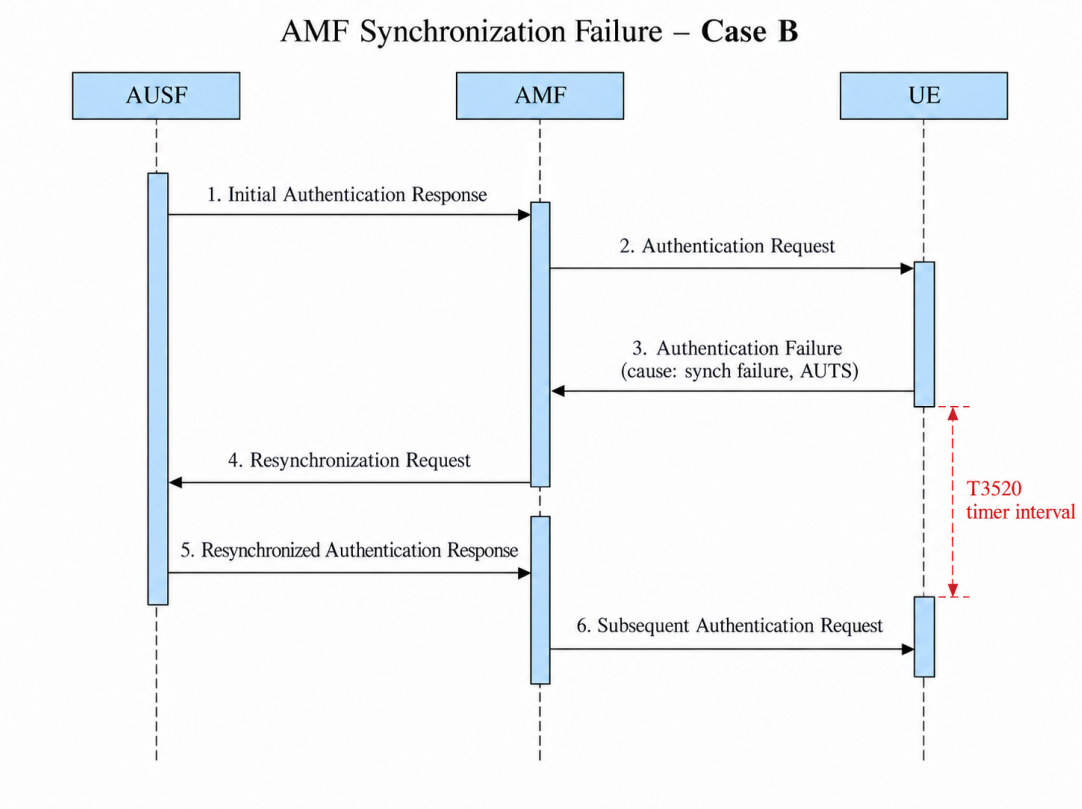


Figura 6.1: Scambio di messaggi nell'AMF Synchronization Failure, Case B, con vincolo temporale T3520.

La specifica DSL del caso descrive la successione degli eventi attesi e i criteri statici da valutare:

```
SEQUENCE {
    ausf.authentication.response AS initial_vector
    nas.authentication_request AS initial_auth
    nas.authentication_failure AS sync_failure
    ausf.resynchronization.request AS resync
    ausf.authentication.response AS new_vector
    nas.authentication_request AS subsequent_auth_request
}

REQUIRE initial_auth.rand == initial_vector.rand
REQUIRE sync_failure.cause == 21
REQUIRE resync.auts == sync_failure.auts
REQUIRE resync.rand == initial_auth.rand
```

A questa si affianca la proprietà temporale MFOTL, che esclude la presenza di una nuova `Authentication Request` precedente alla risposta AUSF nell'intervallo configurato dopo il fallimento di sincronizzazione:

```
MFOTL CASE B """
  SyncFailure()
  IMPLIES
  NOT (
      (NOT AusfAuthenticationResponse())
      UNTIL[0,{TIMER_MS}]
      AuthenticationRequest()
  )
"""
```

L'analisi delle catture provenienti da free5GC e Open5GS ha prodotto, in entrambi i casi, un unico match completo, associando agli alias dichiarati gli eventi individuati nel traffico. Le quattro verifiche statiche hanno restituito PASS per entrambe le implementazioni:

- corrispondenza del `RAND` tra risposta `AUSF` iniziale e l'Authentication Request;
- presenza della causa `NAS 21` nell'Authentication Failure;
- corrispondenza di `AUTS` tra l'Authentication Failure e la richiesta di risincronizzazione;
- corrispondenza del `RAND` iniziale con quello utilizzato nella richiesta di risincronizzazione.

Anche la verifica temporale ha restituito PASS in entrambe le catture, sebbene siano stati osservati intervalli differenti tra i messaggi. In free5GC, l'Authentication Failure viene individuato al frame 358, mentre la successiva risposta AUSF e la nuova Authentication Request compaiono rispettivamente ai frame 604 e 606, circa 25 secondi dopo il fallimento di sincronizzazione. In Open5GS, gli stessi eventi vengono individuati ai frame 276, 432 e 434, con un intervallo di circa 11 secondi tra l'Authentication Failure e la nuova Authentication Request.

Nonostante la differenza negli intervalli osservati, in entrambe le catture la nuova Authentication Request segue la risposta AUSF. La verifica temporale restituisce quindi `PASS` per entrambe le implementazioni e, poiché anche i quattro criteri statici risultano soddisfatti, il verdetto complessivo è `PASS`. Il risultato mostra come la stessa

specifica DSL e gli stessi componenti di verifica possano essere applicati a catture provenienti da implementazioni differenti, valutando sia la coerenza dei parametri scambiati sia le relazioni temporali tra gli eventi.

6.3.2 UPF TEID Uniqueness

Il test `UPF TEID Uniqueness` impiega i costrutti `COLLECT` e `REQUIRE UNIQUE` per verificare l'unicità degli `F-TEID`, identificatori utilizzati per distinguere i tunnel GTP-U associati al traffico del piano utente. Durante l'instaurazione delle sessioni PFCP, l'UPF comunica tali identificatori attraverso i messaggi `PFCP Session Establishment Response`. La verifica considera gli `F-TEID` estratti dalle risposte osservate, controllando che non vengano riutilizzati tra eventi distinti.

La specifica DSL utilizzata è la seguente:

```
COLLECT pfcp.session_establishment_response AS responses

REQUIRE UNIQUE responses.f_teids
```

La prima istruzione raccoglie tutti gli eventi corrispondenti al tipo semantico indicato, associandoli all'alias `responses`. La seconda applica il criterio di unicità ai valori `F-TEID` contenuti nella collezione, rappresentati come coppie strutturate `{teid, ipv4}`.

Nella cattura analizzata, il sistema ha individuato cinque risposte PFCP, corrispondenti ai frame 37, 38, 46, 47 e 48, dalle quali sono stati estratti complessivamente dieci F-TEID, due per ciascuna risposta. Pur condividendo lo stesso indirizzo IPv4, gli identificatori presentano valori TEID differenti; non essendo stata rilevata alcuna coppia `{teid, ipv4}` duplicata tra eventi distinti, il verificatore restituisce il verdetto complessivo `PASS`.

6.3.3 AMF RES* Verification Failure - Cases A/B/C/D

Il test `AMF RES* Verification Failure` riguarda la gestione di un fallimento nella verifica della risposta di autenticazione `RES*` durante la procedura 5G AKA. Le quattro varianti considerate differiscono per il percorso di identificazione dell'UE e per l'origine del fallimento. L'identificazione può avvenire direttamente tramite SUCI oppure attraverso un'identificazione iniziale mediante GUTI, seguita da una richiesta d'identità; il fallimento può invece essere rilevato localmente dall'AMF oppure comunicato dall'AUSF.

Per rappresentare queste varianti mediante un'unica specifica DSL, vengono utilizzati i blocchi `OPTIONAL` per gli scambi di messaggi la cui presenza dipende dal percorso considerato e le regole `IF` per applicare i criteri di verifica corrispondenti alla configurazione del caso di test.

Le combinazioni considerate sono riportate nella Tabella 6.3.

Caso	Identificazione	Origine del fallimento	Eventi opzionali presenti
A	SUCI diretto	AMF	Nessuno
B	GUTI → SUCI	AMF	Identity Request/Response
C	SUCI diretto	AUSF	Confirmation Request/Response
D	GUTI → SUCI	AUSF	Entrambi i gruppi

Tabella 6.3: Varianti del test AMF RES* Verification Failure

La specifica DSL utilizza un'unica `SEQUENCE` contenente due blocchi `OPTIONAL`, rispettivamente dedicati agli scambi di identificazione e alla conferma dell'autenticazione con l'AUSF.

```
SEQUENCE {
    nas.registration_request AS registration_request

    OPTIONAL {
        nas.identity_request AS identity_request
        nas.identity_response AS identity_response
    }

    ausf.authentication.response AS authentication_vector
    nas.authentication_request AS authentication_request
    nas.authentication_response AS authentication_response

    OPTIONAL {
        ausf.confirmation.request AS ausf_confirmation_request
        ausf.confirmation.response AS ausf_confirmation_response
    }

    nas.authentication_reject AS authentication_reject
}
```

I criteri di verifica comprendono tre regole comuni, dedicate alla coerenza dei parametri `RAND` e `AUTN` e alla presenza dell'`Authentication Reject` finale. A queste si aggiungono le verifiche specifiche del percorso di identificazione dell'UE e dell'origine del fallimento, applicate in funzione della variante considerata.

```
REQUIRE authentication_vector.rand == authentication_request.rand
REQUIRE authentication_vector.autn == authentication_request.autn
REQUIRE authentication_reject.present == true

IF case.identity_path == direct_suci {
    REQUIRE registration_request.identity_type == SUCI
}

IF case.identity_path == guti_then_suci {
    REQUIRE registration_request.identity_type == 5G_GUTI
    REQUIRE identity_request.requested_identity == SUCI
    REQUIRE identity_response.identity_type == SUCI
}

IF case.failure_source == local {
    REQUIRE hres_star.value != authentication_vector.hxres_star
}

IF case.failure_source == ausf {
    REQUIRE hres_star.value == authentication_vector.hxres_star
    REQUIRE authentication_response.res_star == ausf_confirmation_request.
  ↪ res_star
    REQUIRE ausf_confirmation_response.auth_result == AUTHENTICATION_FAILURE
}
```

Oltre ai binding prodotti dal matcher attraverso il riconoscimento degli eventi della sequenza, la specifica utilizza informazioni aggiuntive fornite al verificatore. In particolare, `case.identity_path` e `case.failure_source` sono *binding di configurazione* che identificano rispettivamente il percorso di identificazione e l'origine del fallimento previsti dalla variante del test, mentre `hres_star.value` è un *binding derivato*, calcolato a partire dai parametri di autenticazione. Questi valori, pur non essendo ottenuti direttamente dal matcher, vengono integrati nel `BindingContext` insieme agli eventi selezionati, permettendo alla DSL di utilizzarli nelle regole di verifica.

L'analisi delle quattro catture free5GC ha prodotto un match completo per ciascuna variante, riconoscendo la presenza o l'assenza dei gruppi di eventi opzionali. I risultati delle verifiche sono riportati nella tabella seguente.

Caso	PASS	SKIPPED	Totale	Verdetto
A	7	2	9	PASS
B	9	2	11	PASS
C	9	2	11	PASS
D	11	2	13	PASS

Tabella 6.4: Risultati delle verifiche AMF RES* sulle catture free5GC. I conteggi includono anche i risultati aggregati dei blocchi IF.

La differenza nel numero di criteri riportati dipende dalle verifiche applicabili a ciascuna variante. Tutti i casi condividono tre regole `REQUIRE` iniziali e quattro condizioni `IF`, ma il numero di regole interne eseguite varia in funzione del percorso

osservato. Nel Case A, ad esempio, l'identificazione diretta tramite `SUCI` e il fallimento locale richiedono una verifica aggiuntiva ciascuno, mentre nel Case D il percorso `GUTI` → `SUCI` e il fallimento comunicato dall'AUSF comportano tre verifiche aggiuntive per ciascuno dei due aspetti.

Confronto con le catture Open5GS. Una seconda verifica è stata eseguita su quattro catture Open5GS riferite alle stesse varianti RES*, applicando la medesima DSL e il verificatore comune. I risultati sono riportati nella Tabella 6.5.

Caso	Evidenza selezionata	Verdetto
A	Match completo e criteri statici soddisfatti	PASS
B	Match completo; assenti i binding per Identity Request/Response	INCONCLUSIVE
C	Nessun match completo	INCONCLUSIVE
D	Nessun match completo	INCONCLUSIVE

Tabella 6.5: Verifica delle varianti AMF RES* su catture Open5GS.

Per comprendere i risultati `INCONCLUSIVE` ottenuti nei Case B, C e D, è necessario distinguere le evidenze individuate durante la selezione. Nel Case B, il matcher riconosce una sequenza completa, associando agli eventi principali della procedura i frame 617, 624, 626, 627 e 628; tuttavia, gli alias `identity_request` e `identity_response`, appartenenti al blocco `OPTIONAL` dedicato all'identificazione, rimangono privi di un evento associato. Poiché tali binding sono richiesti dai criteri condizionali applicabili al percorso `GUTI` → `SUCI`, la verifica non può essere completata. Nei Case C e D, invece, il matcher non individua alcuna sequenza completa, impedendo già nella fase di selezione l'acquisizione delle evidenze necessarie per procedere alla verifica statica.

Questi risultati evidenziano due situazioni distinte. Nel Case B, il riconoscimento completo del pattern non garantisce la disponibilità di tutti i binding richiesti dai criteri applicabili, mentre nei Case C e D l'assenza di un match completo impedisce di avviare la verifica.

6.4 Mutation testing

Le mutazioni sono state introdotte nelle catture originali mediante procedure predisposte in funzione dei criteri definiti nelle rispettive specifiche DSL. Tali procedure appartengono all'infrastruttura sperimentale e operano sulle evidenze osservate, senza modificare l'interpretazione delle regole o la logica del verificatore.

Le catture originali e quelle modificate vengono elaborate attraverso gli stessi meccanismi di selezione e verifica, così da confrontare i verdetti ottenuti e valutare la risposta del sistema alle alterazioni introdotte. La Tabella 6.6 riporta, per ciascuno scenario, la modifica effettuata e i risultati ottenuti prima e dopo la mutazione.

Caso di test	**Alterazione introdotta**	**Baseline**	**Mutato**
NRF Discovery Authorization	Status HTTP da 200 a 403	FAIL	PASS
UDM Resynchronization	Alterazione del parametro RAND	PASS	FAIL
UDM SUCI De-concealment	Alterazione del SUPI restituito	PASS	FAIL
AMF Sync Failure – Case A	Alterazione di RAND nella richiesta di risincronizzazione	PASS	FAIL
	Modifica dei timestamp per violare il vincolo temporale T3520		
	Combinazione delle alterazioni statica e temporale		
AMF Sync Failure – Case B	Alterazione di RAND nella richiesta di risincronizzazione	PASS	FAIL
UPF TEID Uniqueness	Introduzione di una coppia F-TEID duplicata	PASS	FAIL
	Rimozione delle risposte PFCP rilevanti		INCONCLUSIVE
AMF RES* – Case A	Alterazione del parametro HXRES* nel confronto locale	PASS	FAIL
AMF RES* – Case B	Alterazione del parametro HXRES* nel confronto locale	PASS	FAIL
AMF RES* – Case C	Esito AUSF modificato da FAILURE a SUCCESS	PASS	FAIL
AMF RES* – Case D	Esito AUSF modificato da FAILURE a SUCCESS	PASS	FAIL

Tabella 6.6: Risultati delle mutazioni applicate ai casi di test SCAS.

Le mutazioni relative ai test NRF, UDM, AMF Synchronization Failure e UPF, insieme a quella del Case A di AMF RES* Verification Failure, sono state valutate su catture Open5GS. Le mutazioni dei Case B, C e D di AMF RES* Verification Failure si riferiscono invece a catture free5GC. Questa distinzione permette di identificare l'origine delle evidenze utilizzate nei diversi esperimenti.

Per il test AMF Synchronization Failure – Case A sono state considerate tre mutazioni della cattura Open5GS, relative rispettivamente ai valori dei parametri di autenticazione, agli intervalli temporali e alla combinazione delle due alterazioni. La Tabella 6.7 permette di distinguere, per ciascuna variante, il risultato della verifica statica da quello della verifica temporale.

Mutazione	**Statico**	**Temporale**	**Complessivo**
RAND	FAIL	PASS	FAIL
Timestamp	PASS	FAIL	FAIL
Combinata	FAIL	FAIL	FAIL

Tabella 6.7: Risultati delle verifiche statica e temporale nelle mutazioni AMF Synchronization Failure – Case A su Open5GS.

Quando viene alterato il parametro `RAND`, il verificatore statico rileva l'incoerenza tra i valori associati agli eventi, mentre il vincolo temporale rimane soddisfatto. Viceversa, modificando gli intervalli temporali tra gli eventi, le condizioni statiche rimangono valide, ma la formula `MFOTL` non risulta soddisfatta. La combinazione delle due mutazioni produce il fallimento di entrambi i componenti. Questo comportamento evidenzia come le verifiche statiche e temporali possano individuare violazioni differenti sulla stessa evidenza.

Un secondo risultato rilevante emerge dal test `UPF TEID Uniqueness`. L'introduzione di un F-TEID duplicato determina un verdetto `FAIL`, poiché viene violato il requisito di unicità, mentre la rimozione delle risposte PFCP necessarie alla verifica produce `INCONCLUSIVE`, in quanto il sistema non dispone dell'evidenza sufficiente per valutare il criterio.

Complessivamente, nei 13 scenari di mutazione considerati, i verdetti osservati corrispondono a quelli attesi. I risultati forniscono evidenza sperimentale della capacità del sistema di rilevare le alterazioni introdotte, utilizzando gli stessi meccanismi di selezione e verifica applicati alle catture originali. Tale valutazione rimane tuttavia circoscritta ai casi e alle mutazioni esaminati e non costituisce una dimostrazione di correttezza generale del verificatore.

Capitolo 7

Discussione, conclusioni e sviluppi futuri

I risultati ottenuti sui casi di test esaminati permettono di rispondere positivamente alla domanda posta all'inizio del lavoro: è possibile rappresentare in forma interpretabile i criteri necessari a verificare l'esito di un test SCAS a partire dal traffico osservato. La sperimentazione ha mostrato che tale approccio può essere applicato a requisiti differenti senza dover sviluppare una logica di verifica dedicata per ogni caso, pur richiedendo una fase di interpretazione della specifica per individuare le proprietà effettivamente verificabili.

L'applicazione delle medesime specifiche DSL a catture provenienti da Open5GS e free5GC ha inoltre permesso di valutare la riusabilità della pipeline su implementazioni differenti della 5G Core Network.

Un aspetto emerso dal lavoro è che la definizione di un criterio e la possibilità di valutarlo sono due questioni distinte. Anche quando una proprietà può essere espressa nella DSL, la sua verifica dipende dalla disponibilità di evidenze sufficienti nella cattura e dalla possibilità di ricavarne le informazioni necessarie.

Queste considerazioni definiscono anche i limiti dei risultati raggiunti. La DSL supporta attualmente un insieme circoscritto di proprietà e la validazione riguarda un numero selezionato di casi SCAS e di mutazioni controllate. Non è pertanto possibile concludere che qualsiasi requisito di sicurezza possa essere verificato attraverso il solo traffico di rete, né che i risultati ottenuti siano generalizzabili a ogni possibile scenario.

Un limite dell'attuale implementazione riguarda la dipendenza della verifica temporale dalla selezione delle evidenze. Poiché il log fornito a MonPoly viene costruito a partire dagli eventi riconosciuti dal matcher, una violazione dell'ordinamento richiesto dalla SEQUENCE può impedire l'esecuzione della verifica temporale che dovrebbe rilevarla. Una possibile direzione di sviluppo consiste quindi nel rendere tale verifica meno dipendente dal completamento del riconoscimento sequenziale.

Il limite della verifica temporale appena descritto mostra come l'analisi dei risultati sperimentali possa individuare aspetti dell'implementazione che richiedono ulteriori sviluppi. Seguendo questo approccio, l'estensione del sistema potrebbe essere guidata dall'esame di nuovi requisiti SCAS, individuando le proprietà non ancora rappresentabili attraverso il DSL e le informazioni che gli attuali normalizzatori non consentono di ricavare. Le eventuali estensioni del linguaggio e dei meccanismi di estrazione

potrebbero così rispondere a esigenze di verifica concrete, mantenendo la riusabilità dei componenti comuni.

Lo stesso approccio potrebbe contribuire anche all'evoluzione del *mutation testing*, attualmente basato su procedure di alterazione predisposte per i singoli scenari sperimentali. L'utilizzo delle specifiche DSL per individuare gli eventi e i campi interessati dalle mutazioni permetterebbe di riutilizzare parte della logica di selezione delle evidenze, riducendo la necessità di sviluppare procedure interamente dedicate a ciascun caso di test. Le operazioni di modifica rimarrebbero comunque specifiche rispetto ai protocolli e alle proprietà da verificare.

Il lavoro costituisce quindi un primo passo verso una verifica degli esiti SCAS maggiormente sistematica e riutilizzabile, il cui ambito di applicazione rimane strettamente legato alle proprietà esprimibili e all'evidenza effettivamente osservabile.

Bibliografia

[1] 3GPP. *Catalogue of General Security Assurance Requirements*. Rapp. tecn. ETSI TS 133 117. Ver. 15.4.0. 3GPP TS 33.117, Release 15. ETSI, 2019. URL: https://www.etsi.org/deliver/etsi_ts/133100_133199/133117/15.04.00_60/ts_133117v150400p.pdf.

[2] GSMA. *FS.13 – Network Equipment Security Assurance Scheme (NESAS) Overview*. Rapp. tecn. FS.13. Ver. 3.0. 20 February 2025. GSMA, feb. 2025. URL: https://www.gsma.com/solutions-and-impact/technologies/security/wp-content/uploads/2025/04/FS.13-v3.0.pdf.

[3] 3GPP. *System Architecture for the 5G System (5GS)*. Rapp. tecn. ETSI TS 123 501. Ver. 16.15.0. 3GPP TS 23.501, Release 16. ETSI, 2023. URL: https://www.etsi.org/deliver/etsi_ts/123500_123599/123501/16.15.00_60/ts_123501v161500p.pdf.

[4] 3GPP. *Security Architecture and Procedures for 5G System*. Rapp. tecn. ETSI TS 133 501. Ver. 15.11.0. 3GPP TS 33.501, Release 15. ETSI, 2021. URL: https://www.etsi.org/deliver/etsi_ts/133500_133599/133501/15.11.00_60/ts_133501v151100p.pdf.

[5] GSMA. *FS.63 – List of Adopted Security Assurance Specifications*. Rapp. tecn. FS.63. Ver. 3.0. 12 September 2025. GSMA, set. 2025. URL: https://www.gsma.com/solutions-and-impact/technologies/security/wp-content/uploads/2019/03/FS.63-v3.0.pdf.

[6] 3GPP. *Security Assurance Methodology (SECAM) for 3GPP Network Products*. Rapp. tecn. ETSI TR 133 916. Ver. 18.0.0. 3GPP TR 33.916, Release 18. ETSI, 2024. URL: https://www.etsi.org/deliver/etsi_tr/133900_133999/133916/18.00.00_60/tr_133916v180000p.pdf.

[7] 3GPP. *Security Assurance Specification (SCAS) for the Network Repository Function (NRF)*. Rapp. tecn. ETSI TS 133 518. Ver. 18.0.0. 3GPP TS 33.518, Release 18, section 4.2.2.2.1. ETSI, 2024. URL: https://www.etsi.org/deliver/etsi_ts/133500_133599/133518/18.00.00_60/ts_133518v180000p.pdf.

[8] Wireshark Foundation. *TShark Manual Page*. URL: https://www.wireshark.org/docs/man-pages/tshark.html (visitato il giorno 11/10/2026).

[9] K. Thompson. «Regular Expression Search Algorithm». In: *Communications of the ACM* 11.6 (1968), pp. 419–422. DOI: 10.1145/363347.363387.

[10] D. Basin et al. «Monitoring Metric First-Order Temporal Properties». In: *Journal of the ACM* 62.2 (2015). DOI: 10.1145/2699444.

[11] D. Basin, F. Klaedtke e E. Zălinescu. «The MonPoly Monitoring Tool». In: *RV-CuBES 2017*. Vol. 3. Kalpa Publications in Computing. 2017, pp. 19–28. DOI: 10.29007/89hs.

[12] Y. Jia e M. Harman. «An Analysis and Survey of the Development of Mutation Testing». In: *IEEE Transactions on Software Engineering* 37.5 (2011), pp. 649–678. DOI: 10.1109/TSE.2010.62.

[13] Open5GS Project. *Open5GS Documentation*. URL: https://open5gs.org/open5gs/docs/ (visitato il giorno 11/10/2026).

[14] free5GC Project. *free5GC Documentation*. URL: https://free5gc.org/guide/ (visitato il giorno 11/10/2026).

[15] F. Mancini et al. «ScasDK—An All-in-One Test Platform for Security Assurance in 5G Core Networks». In: *Computer Networks* 265 (2025), p. 111296. DOI: 10.1016/j.comnet.2025.111296.

[16] 3GPP. *Security Assurance Specification (SCAS) for the Unified Data Management (UDM)*. Rapp. tecn. ETSI TS 133 514. Ver. 16.4.0. 3GPP TS 33.514, Release 16. ETSI, 2021. URL: https://www.etsi.org/deliver/etsi_ts/133500_133599/133514/16.04.00_60/ts_133514v160400p.pdf.

[17] 3GPP. *Security Assurance Specification (SCAS) for the Access and Mobility Management Function (AMF)*. Rapp. tecn. ETSI TS 133 512. Ver. 16.6.0. 3GPP TS 33.512, Release 16. ETSI, 2022. URL: https://www.etsi.org/deliver/etsi_ts/133500_133599/133512/16.06.00_60/ts_133512v160600p.pdf.

[18] 3GPP. *Security Assurance Specification (SCAS) for the User Plane Function (UPF)*. Rapp. tecn. ETSI TS 133 513. Ver. 16.3.0. 3GPP TS 33.513, Release 16. ETSI, 2023. URL: https://www.etsi.org/deliver/etsi_ts/133500_133599/133513/16.03.00_60/ts_133513v160300p.pdf.

[19] 3GPP. *5G; Non-Access-Stratum (NAS) protocol for 5G System (5GS); Stage 3*. Rapp. tecn. ETSI TS 124 501. Ver. 16.15.0. 3GPP TS 24.501 V16.15.0, Release 16; tabella 10.2.1. ETSI, lug. 2025. URL: https://www.etsi.org/deliver/etsi_ts/124500_124599/124501/16.15.00_60/ts_124501v161500p.pdf.

Glossario e acronimi

Il glossario raccoglie le principali sigle e i termini delle reti 5G utilizzati nel testo, con particolare riferimento all'architettura di rete e alle procedure di autenticazione.

Termine	Definizione
3GPP	3rd Generation Partnership Project: organizzazione di collaborazione internazionale che produce specifiche tecniche per le reti mobili, incluso il sistema 5G.
5G AKA	5G Authentication and Key Agreement: procedura di autenticazione primaria e accordo delle chiavi nel sistema 5G.
5GC	5G Core Network: insieme delle funzioni di rete che costituiscono la rete centrale del sistema 5G.
5G-GUTI (GUTI)	5G Globally Unique Temporary UE Identity: identificativo temporaneo assegnato dalla rete all'UE, utilizzabile nelle procedure di segnalazione senza esporre sistematicamente l'identità permanente.
5GMM cause #21	Synch failure: codice di causa incluso in un messaggio Authentication Failure quando l'UE rileva un problema di sincronizzazione nella verifica della sfida di autenticazione.
AMF	Access and Mobility Management Function: funzione della 5G Core Network che gestisce registrazione, accesso e mobilità dell'UE e coordina alcune procedure di autenticazione.
AUSF	Authentication Server Function: funzione della 5G Core Network coinvolta nell'esecuzione dell'autenticazione primaria dell'utente.
Authentication Failure	Messaggio NAS con cui l'UE segnala alla rete il fallimento di una procedura di autenticazione, indicandone la causa e, quando previsto, ulteriori parametri.
Authentication Reject	Messaggio NAS con cui la rete comunica all'UE il rifiuto dell'autenticazione.
Authentication Request	Messaggio NAS con cui la rete richiede all'UE di eseguire l'autenticazione, trasmettendo i parametri della sfida previsti dalla procedura.
AUTN	Authentication Token: parametro della sfida 5G AKA che consente all'UE di effettuare i controlli di autenticità e freschezza previsti dalla procedura.
AUTS	Parametro di risincronizzazione trasmesso dall'UE quando viene rilevato un problema di sincronizzazione durante l'autenticazione 5G AKA.

Termine	Definizione
F-TEID	Fully Qualified Tunnel Endpoint Identifier: identificazione di un endpoint di tunnel GTP, composta dal TEID e dalle relative informazioni di indirizzamento IP.
GTP-U	GPRS Tunnelling Protocol – User Plane: protocollo utilizzato per trasportare il traffico dati degli utenti attraverso tunnel nel piano utente.
HTTP/2	Protocollo applicativo impiegato nelle comunicazioni basate su servizi fra le funzioni del piano di controllo della 5G Core Network.
HXRES*	Valore ottenuto a partire dalla risposta di autenticazione attesa, utilizzato dall'AMF per il controllo locale della risposta dell'UE nella procedura 5G AKA.
NAS	Non-Access Stratum: insieme di protocolli di segnalazione tra l'UE e la rete centrale 5G, in particolare l'AMF, per procedure quali registrazione e autenticazione.
Network slice	Rete logica definita sul sistema 5G, con funzionalità e risorse configurate per determinate esigenze di servizio.
NF	Network Function: funzione di rete che svolge un compito specifico all'interno dell'architettura 5G.
NF Discovery	Procedura con cui una funzione di rete individua, tramite NRF, altre funzioni e i servizi disponibili necessari a una comunicazione.
NRF	Network Repository Function: funzione che supporta la registrazione, l'aggiornamento e la scoperta delle funzioni di rete e dei loro servizi.
PCAP	Packet Capture: file utilizzato per conservare i pacchetti acquisiti durante una cattura di traffico di rete.
PFCP	Packet Forwarding Control Protocol: protocollo utilizzato per controllare le sessioni e le regole di inoltro del piano utente, nelle comunicazioni con l'UPF.
Piano di controllo	Parte dell'architettura di rete dedicata alla segnalazione e alla gestione delle procedure, delle connessioni e delle risorse.
Piano utente	Parte dell'architettura di rete dedicata al trattamento e al trasporto del traffico dati degli utenti.
RAND	Valore casuale incluso nella sfida di autenticazione, utilizzato come input nei calcoli della procedura 5G AKA.
RES*	Risposta di autenticazione calcolata dall'UE nella procedura 5G AKA e utilizzata nel controllo della risposta ricevuta.
SBA	Service-Based Architecture: modello nel quale le funzioni del piano di controllo espongono e utilizzano servizi mediante interfacce standardizzate.
SCAS	Security Assurance Specification: specifica 3GPP che definisce requisiti di sicurezza e casi di test per una determinata classe di prodotti o funzioni di rete.
SUCI	Subscription Concealed Identifier: rappresentazione protetta dell'identificativo permanente dell'abbonato, utilizzata per limitarne l'esposizione durante le procedure di rete.

Termine	Definizione
SUPI	Subscription Permanent Identifier: identificativo permanente dell'abbonato nel sistema 5G.
T3520	Timer NAS avviato dall'UE in particolari condizioni di fallimento dell'autenticazione, tra cui la causa di sincronizzazione. Il valore ordinario è di 15 secondi; la specifica 3GPP prevede valori differenti per alcune modalità di accesso.
TEID	Tunnel Endpoint Identifier: identificativo utilizzato per distinguere un endpoint di tunnel nelle comunicazioni GTP.
UDM	Unified Data Management: funzione della 5G Core Network che gestisce dati degli abbonati e informazioni utilizzate nelle procedure di autenticazione e servizio.
UE	User Equipment: dispositivo dell'utente che accede alla rete mobile, ad esempio uno smartphone o un terminale con connettività cellulare.
UPF	User Plane Function: funzione della 5G Core Network che tratta e inoltra il traffico dati sul piano utente.